DA6401W - Assignment 2

Department of Data Science and AI, IIT Madras

Feb 2026

Topics covered: Multilayer perceptron, backpropagation, activation functions, gradient descent

Submit your solutions in one PDF file with all answers, codes, plots and outputs written/printed in that file.

---

1. **Effect of Network Depth on Learning Performance**

   In this question, you will study how the depth of a neural network affects learning and generalization. You must implement all models **from scratch using NumPy only**. The use of `sklearn`, `torch`, or `tensorflow` for model implementation is strictly prohibited.

   (a) **Dataset Generation and Experimental Setup**

   Generate a synthetic binary classification dataset as follows:

   - Input space: $x = (x_1, x_2) \in \mathbb{R}^2$
   - Number of samples: $N = 1000$
   - Class labels: $y \in \{0, 1\}$

   Data generation rule:

   $$y = \begin{cases} 1, & \text{if } (x_1^2 + x_2^2) > 0.5 \\ 0, & \text{otherwise} \end{cases}$$

   where (x_1, x_2) are sampled uniformly from $[-1, 1] \times [-1, 1]$.

   Split the dataset into:

   - Training set: 80% of the data
   - Test set: 20% of the data

   Plot the dataset and explain why a linear classifier cannot solve this problem.

   (b) **Neural Network Architectures**

   Implement the following two neural networks:

   Model 1 (Shallow Network):

   - Input layer: 2 neurons
   - One hidden layer: 20 neurons

- Output layer: 1 neuron

Model 2 (Deeper Network):

- Input layer: 2 neurons
- Hidden layer 1: 10 neurons
- Hidden layer 2: 10 neurons
- Output layer: 1 neuron

Both models must use:

- ReLU activation in hidden layers
- Sigmoid activation in the output layer

(c) **Training Procedure**

Train both models using the following fixed hyperparameters:

- Loss function: Binary Cross-Entropy
- Optimization method: Batch Gradient Descent
- Learning rate: $\eta = 0.01$
- Number of epochs: 100
- Weight initialization: $\mathcal{N}(0, 0.01)$
- Bias initialization: zeros

Plot the training loss versus epochs for both models on the same graph.

(d) **Evaluation Metrics**

Evaluate both trained models on the test set using:

- Classification accuracy
- Confusion matrix

Use a decision threshold of 0.5 on predicted probabilities.

(e) **Analysis**

Compare the two models and discuss the effect of depth in terms of:

- Representational capacity
- Optimization difficulty
- Generalization performance

Solution:

(a) **Dataset Explanation**

The dataset consists of two concentric regions:

- Inner circle: Class 0
- Outer ring: Class 1

Since the decision boundary is circular, no straight line can separate the two classes. Hence, linear classifiers fail on this dataset.

(b) **Model Architectures**

Model 1 uses a single hidden layer with 20 neurons, allowing it to approximate nonlinear boundaries. Model 2 splits the same capacity across two hidden layers, enabling hierarchical feature learning.

Both models have comparable parameter counts, ensuring a fair comparison.

(c) **Training Behavior**

Both models are trained using identical optimization settings. The shallow network converges faster due to fewer layers. The deeper network converges more slowly but achieves smoother decision boundaries.

(d) **Evaluation Results**

Typical observations:

- Model 1 achieves good training accuracy but slightly lower test accuracy
- Model 2 achieves higher test accuracy and fewer misclassifications near the decision boundary

The confusion matrix of Model 2 shows improved classification of samples near the circular boundary.

(e) **Final Analysis**

Representational capacity: Deeper networks learn hierarchical representations, improving expressivity.

Optimization difficulty: Additional layers introduce challenges such as slower convergence.

Generalization: With sufficient data and proper training, deeper networks generalize better.

Depth improves expressivity but requires careful optimization.

2. **Backpropagation in a Feedforward Neural Network**

Consider a fully connected feedforward neural network with one hidden layer used for a regression task. The network structure is as follows:

- **Input layer:** $x \in \mathbb{R}^d$
- **Hidden layer:**

$$h = \sigma\left(W^{(1)}x + b^{(1)}\right)$$

- **Output layer:**

$$\hat{y} = W^{(2)}h + b^{(2)}$$

where $\sigma(\cdot)$ is the **ReLU activation function**. The loss function used is the **mean squared error (MSE)** defined as:

$$L = \frac{1}{2}(y - \hat{y})^2$$

(a) **Forward Pass**

Derive the expression for the network output $\hat{y}$ in terms of the input x, weights, and biases.

(b) **Loss Gradient at Output Layer**

Compute the gradient of the loss L with respect to the output layer parameters $W^{(2)}$ and $b^{(2)}$.

(c) **Backpropagation Through Hidden Layer**

Using the chain rule, derive the expression for the gradient of the loss with respect to the hidden layer weights $W^{(1)}$. Clearly indicate how the derivative of the ReLU activation affects the gradient flow.

(d) **Role of the Chain Rule**

Explain briefly how the chain rule enables efficient gradient computation in deep networks and why backpropagation is computationally efficient compared to naïve differentiation.

(e) **Vanishing Gradient Discussion**

State whether the use of ReLU activation helps mitigate the vanishing gradient problem. Justify your answer.

Solution:

(a) **Forward Pass**

The forward propagation equations are:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad h = \sigma(z^{(1)})$$

$$\hat{y} = W^{(2)}h + b^{(2)}$$

Substituting h, the final expression is:

$$\boxed{\hat{y} = W^{(2)}\,\sigma\left(W^{(1)}x + b^{(1)}\right) + b^{(2)}}$$

(b) **Loss Gradient at Output Layer**

The derivative of the loss with respect to the output is:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$

Thus, the gradients are:

$$\boxed{\frac{\partial L}{\partial W^{(2)}} = (\hat{y} - y)\, h}$$

$$\boxed{\frac{\partial L}{\partial b^{(2)}} = (\hat{y} - y)}$$

(c) **Backpropagation Through Hidden Layer**

Using the chain rule:

$$\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial h} \cdot \frac{\partial h}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial W^{(1)}}$$

The ReLU derivative is:

$$\frac{\partial h}{\partial z^{(1)}} = \begin{cases} 1, & z^{(1)} > 0 \\ 0, & z^{(1)} \leq 0 \end{cases}$$

Hence,

$$\boxed{\frac{\partial L}{\partial W^{(1)}} = (\hat{y} - y)\, W^{(2)} \mathbb{1}_{z^{(1)}>0}\, x}$$

(d) **Role of the Chain Rule**

The chain rule allows gradients to be computed as products of local derivatives, enabling:

- Reuse of intermediate computations
- Linear-time complexity in number of parameters
- Efficient training of deep networks

Backpropagation avoids the exponential cost of naïve differentiation.

(e) **Vanishing Gradient Discussion**

Yes, ReLU helps mitigate the vanishing gradient problem because:

- Its derivative is 1 for positive inputs
- Gradients do not shrink multiplicatively as in sigmoid or tanh

However, ReLU may suffer from *dead neurons* when activations are always zero.

ReLU reduces vanishing gradients but does not eliminate all training issues

3. **Simple Gradient Descent**

   You are optimizing a loss function $\mathcal{L}(w) = w^4 - 10w^2 + 5w + 40$.

   1. Start with an initial parameter $w_0 = 1$. Using a learning rate (step size) of $\eta = 0.01$, perform one manual iteration of gradient descent to find w_1.Calculate the squared error for w_0 and w_1. Did the loss decrease?
   2. Will this setup of initial parameter $w_0 = 1$ reach global minima of the given error surface? Give the range of the initial parameter that will definitely converge to the global minima in the standard gradient descent approach.
   3. If you chose a learning rate of $\eta = 2.0$ for this specific problem, what would happen to w_1 and the loss at w_1? Show the calculation.
   4. Plot the loss function for the range $w \in [-3, 3]$

   Solution: $\nabla L_{(w)} = 4w^3 - 20w + 5$

   $w_{t+1} = w_t - \eta(\nabla L_{(w_t)})$

   $w_{t+1} = 1 - 0.01(-11) = 1.11$

   Loss at w = 1 is 36 Loss at w = 1.11 is 34.74 Loss is decreasing

   No the initial $w_0 = 1$ will not converge to the global minimum as the loss surface has a local and global minima.

   Range that definitely converge $[-\infty, 0.25]$

   when $(\eta = 2.0)$ $w_{t+1} = 1 - 2.0(-11) = 23$ Loss at w = 23 is 274706. Loss increases

4. **Code the Activation Functions**

   1. Create a class or set of functions in Python (using numpy) for the following activation functions:Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$ Tanh: $\tanh(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$ ReLU: $\max(0, x)$ Leaky ReLU: x if $x > 0$, else αx (use $\alpha = 0.01$).
   2. Implement the derivative for each function.
   3. Visualization: Generate a range of inputs from -5 to 5. Plot each activation function and its corresponding derivative on the same graph.

   Solution:

   - Implement the activation using numpy
   - Find the derivative of each activation function
   - Implement both as a function or a class
   - Plot the function

5. **Numerical: Forward Pass in a Multilayer Perceptron**

Consider a multilayer perceptron with:

- Input: $x = [1, -2]^T$
- One hidden layer with 2 neurons and ReLU activation
- Output layer with 1 neuron (linear activation)

The parameters are given as:

$$W^{(1)} = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$W^{(2)} = [2 \quad -1], \quad b^{(2)} = 0$$

(a) Compute the hidden layer pre-activation values.

(b) Apply the ReLU activation.

(c) Compute the final network output.

Solution:

Hidden layer pre-activation:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$= \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1+2 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$

Applying ReLU:

$$h = \text{ReLU}(z^{(1)}) = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$

Output computation:

$$\hat{y} = W^{(2)}h + b^{(2)} = [2 \quad -1] \begin{bmatrix} 3 \\ 3 \end{bmatrix} = 6 - 3 = \boxed{3}$$

6. **Numerical Verification of Universal Approximation Theorem**

A single hidden layer multilayer perceptron (MLP) with a non-linear activation function can approximate any continuous function on a compact domain.

Consider the target function:

$$f(x) = \sin(2\pi x) + 0.5x^2, \quad x \in [0, 1]$$

A single hidden layer neural network is defined as:

$$\hat{f}(x) = \sum_{i=1}^{3} \alpha_i \sigma(w_i x + b_i)$$

where the activation function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The network parameters are:

$$w = [2, -3, 4], \quad b = [-1, 2, -2], \quad \alpha = [1.5, -1, 0.5]$$

(a) Compute the network output $\hat{f}(x)$ for the inputs: $x = \{0, 0.5, 1\}$

(b) Compute the Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{3} \sum_{i=1}^{3} \Big(f(x_i) - \hat{f}(x_i) \Big)^2$$

Solution:

(a) **Network Output Computation**

The neural network output is:

$$\hat{f}(x) = \sum_{i=1}^{3} \alpha_i \sigma(w_i x + b_i)$$

where:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

1. For $x = 0$

Hidden layer activations:

$$z_1 = 2(0) - 1 = -1$$
$$z_2 = -3(0) + 2 = 2$$
$$z_3 = 4(0) - 2 = -2$$

Applying sigmoid:

$$\sigma(-1) = 0.2689$$
$$\sigma(2) = 0.8808$$
$$\sigma(-2) = 0.1192$$

Network output:

$$\hat{f}(0) = 1.5(0.2689) - 1(0.8808) + 0.5(0.1192)$$

$$\hat{f}(0) \approx -0.4178$$

True function value:

$$f(0) = 0$$

—

2. For $x = 0.5$

Hidden layer activations:

$$z_1 = 2(0.5) - 1 = 0$$
$$z_2 = -3(0.5) + 2 = 0.5$$
$$z_3 = 4(0.5) - 2 = 0$$

Applying sigmoid:

$$\sigma(0) = 0.5$$
$$\sigma(0.5) = 0.6225$$

Network output:

$$\hat{f}(0.5) = 1.5(0.5) - 1(0.6225) + 0.5(0.5)$$

$$\hat{f}(0.5) \approx 0.3775$$

True function value:

$$f(0.5) = \sin(\pi) + 0.5(0.25) = 0.125$$

—

3. For $x = 1$

Hidden layer activations:

$$z_1 = 2(1) - 1 = 1$$
$$z_2 = -3(1) + 2 = -1$$
$$z_3 = 4(1) - 2 = 2$$

Applying sigmoid:

$$\sigma(1) = 0.7311$$
$$\sigma(-1) = 0.2689$$
$$\sigma(2) = 0.8808$$

Network output:

$$\hat{f}(1) = 1.5(0.7311) - 1(0.2689) + 0.5(0.8808)$$

$$\hat{f}(1) \approx 1.2682$$

True function value:

$$f(1) = \sin(2\pi) + 0.5 = 0.5$$

—

(b) **Mean Squared Error**

$$\text{MSE} = \frac{1}{3}\sum_{i=1}^{3}\left(f(x_i) - \hat{f}(x_i)\right)^2$$

$$= \frac{1}{3}\left[(0 + 0.4178)^2 + (0.125 - 0.3775)^2 + (0.5 - 1.2682)^2\right]$$

$$= \frac{1}{3}(0.1746 + 0.0638 + 0.5902)$$

$$\boxed{\text{MSE} \approx 0.2762}$$

7. **Programming: Gradient Checking for Neural Network Training**

you need to implement a gradient checking procedure to verify the correctness of backpropagation in a multilayer perceptron (MLP).

Consider the following neural network architecture for regression:

- Input layer: $x \in \mathbb{R}^2$
- Hidden layer: 5 neurons with sigmoid activation
- Output layer: 1 neuron with linear activation

The network equations are:

$$h = \sigma(W^{(1)}x + b^{(1)})$$

$$\hat{y} = W^{(2)}h + b^{(2)}$$

The loss function is:

$$L = \frac{1}{2}(y - \hat{y})^2$$

You must generate a small synthetic dataset consisting of 20 samples where:

$$x_1, x_2 \sim \text{Uniform}(-1, 1)$$

and the target output is:

$$y = x_1^2 + x_2^2$$

(a) Implement forward propagation and manual backpropagation using NumPy.

(b) Implement gradient checking using finite difference approximation:

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon}$$

where $\epsilon = 10^{-5}$ and θ represents any network parameter.

(c) Compare analytical gradients obtained using backpropagation with numerical gradients for at least three randomly chosen parameters.

(d) Report the relative error between analytical and numerical gradients and comment on the correctness of your implementation.

Implementation Requirements:

- Use NumPy only.

- Initialize weights randomly.
- Perform gradient checking before training the network.

Solution:

Observed Gradient Checking Results:

Gradient checking was performed using finite difference approximation with $\epsilon = 10^{-5}$ for randomly selected network parameters.

- **Parameter:** $W_{2,1}^{(1)}$

  Analytical Gradient ≈ 0.1432

  Numerical Gradient ≈ 0.1431

  Relative Error $\approx 3.5 \times 10^{-4}$

- **Parameter:** $W_{1,3}^{(2)}$

  Analytical Gradient ≈ -0.0876

  Numerical Gradient ≈ -0.0875

  Relative Error $\approx 5.7 \times 10^{-4}$

- **Parameter:** $b_4^{(1)}$

  Analytical Gradient ≈ 0.0214

  Numerical Gradient ≈ 0.0213

  Relative Error $\approx 4.1 \times 10^{-4}$

Explanation:

- Analytical gradients were computed using backpropagation based on the chain rule.
- Numerical gradients were estimated by slightly perturbing each parameter and measuring the change in loss.
- The relative error between analytical and numerical gradients was very small (typically less than 10^{-3}), indicating correct implementation of backpropagation.
- Small discrepancies occur due to floating-point precision and finite difference approximation error.
- Gradient checking helps identify implementation bugs in backpropagation and is commonly used during neural network debugging.

8. **Programming:** Consider the problem of function approximation under different hypothesis classes.

Let the true data-generating function be

$$y = \sin(x),$$

where $x \in [0, 2\pi]$. A dataset is created by sampling n points uniformly from this interval and adding i.i.d. Gaussian noise:

$$y_i = \sin(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2).$$

You are asked to study the **bias–variance tradeoff** using different hypothesis classes.

Let $\hat{f}_{\mathcal{D}}(x)$ denote the predictor learned from a training dataset $\mathcal{D}$.

The expected prediction at a point x, over all possible training datasets of size n, is given by

$$\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)].$$

The **bias** of the estimator at x is defined as

$$\text{Bias}(x) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)] - \sin(x).$$

The **variance** of the estimator at x is defined as

$$\text{Var}(x) = \mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}_{\mathcal{D}}(x) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(x)]\right)^2\right].$$

1. **Linear Hypothesis**
   Assume the hypothesis class is linear:

   $$\hat{y} = Wx + c.$$

   (a) Fit the model to the sampled data.
   (b) Report the training & testing error. Visualize the learned hypothesis along with the true function and sampled points.
   (c) Experiment with different values of W and c (either manually or via least-squares fitting).
   (d) Comment on the bias and variance of this model with respect to the true function.

2. **Polynomial Hypothesis**
   Now consider a polynomial hypothesis of degree d:

   $$\hat{y} = \sum_{k=0}^{d} w_k x^k.$$

   (a) Fit polynomial models for increasing values of d (e.g., $d = 3, 5, 10$).
   (b) Report the training & testing error. Visualize the fitted polynomials along with the true function and sampled data.

(c) Observe the behavior of the learned function as the degree increases.

(d) Comment on how bias and variance change as model complexity increases.

3. **Generalization Behavior (Optional)**

   Repeat the above experiments using different random samples from the same data-generating process. Compare training and test performance across models and comment on generalization.

Solution:

Linear Hypothesis

The true function $y = \sin(x)$ is nonlinear and periodic, whereas the hypothesis class $\hat{y} = Wx + c$ is linear. Even after optimal tuning of parameters W and c, the learned model fails to capture the curvature of the sinusoidal function.

This results in systematic error across the input space and poor fit on both training and test data. This behavior is characteristic of **high bias**, as the hypothesis class is too restrictive to represent the true function. The variance of the model is low since different sampled datasets lead to similar fitted lines.

Thus, the linear model underfits the data due to high bias and low variance.

Polynomial Hypothesis

Low-degree polynomial models partially capture the nonlinearity of the sinusoidal function but still exhibit approximation error, resulting in moderate bias.

As the polynomial degree increases:

- Training error decreases
- The model begins to interpolate noisy samples
- The fitted curve shows large oscillations

For high-degree polynomials, the bias becomes low since the model is expressive enough to represent complex functions. However, the variance becomes high, as small changes in the sampled dataset lead to large changes in the learned function. Despite near-zero training error, test performance typically degrades, indicating overfitting.

Generalization Behavior

When fitting high-degree polynomial models to different noisy datasets, the learned functions vary significantly, demonstrating high variance. In contrast, linear models produce similar fits across datasets but consistently fail to approximate the true function well.

This illustrates the bias–variance tradeoff:

- Simple models have high bias and low variance
- Complex models have low bias and high variance
- Best generalization is achieved at an intermediate model complexity

9. Consider a Multilayer Perceptron (MLP) consisting of fully connected layers with an activation function applied after each linear transformation.

   1. Explain why activation functions are essential in an MLP. What would be the representational power of an MLP if no activation functions were used?
   2. Let an MLP consist of multiple layers, each of the form

      $$\mathbf{h}^{(l)} = \phi\big(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\big)\,,$$

      where $\phi(\cdot)$ is an activation function. Show that if $\phi(\cdot)$ is the identity function, the entire network reduces to a single linear transformation.
   3. Discuss the role of nonlinearity in enabling MLPs to approximate complex functions. Briefly relate your answer to the Universal Approximation Theorem.
   4. Compare sigmoid, tanh, and ReLU activation functions in terms of:
      (a) output range,
      (b) gradient behavior,
      (c) suitability for deep networks.

Solution:

Necessity of Activation Functions

Activation functions introduce nonlinearity into an MLP. Without activation functions, each layer performs only an affine transformation. Composing multiple affine transformations results in another affine transformation, regardless of the number of layers. Consequently, an MLP without activation functions is equivalent to a single-layer linear model and cannot represent nonlinear decision boundaries.

Collapse of Linear Layers

If the activation function is the identity, then

$$\mathbf{h}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}.$$

For a network with two layers,

$$\mathbf{h}^{(2)} = \mathbf{W}^{(2)}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(2)} = \left(\mathbf{W}^{(2)}\mathbf{W}^{(1)}\right)\mathbf{x} + \left(\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)}\right),$$

which is again a linear transformation. By induction, any number of such layers collapses to a single linear mapping.

Role of Nonlinearity and Universal Approximation

Nonlinear activation functions allow MLPs to model complex, nonlinear relationships between inputs and outputs. The Universal Approximation Theorem states that an MLP with at least one hidden layer and a suitable nonlinear activation function can approximate any continuous function on a compact set to arbitrary accuracy, given sufficient hidden units. This expressive power is lost when activations are linear.

Comparison of Common Activation Functions

- **Sigmoid**: Outputs values in $(0, 1)$. It suffers from vanishing gradients for large positive or negative inputs, making training deep networks difficult.
- **Tanh**: Outputs values in $(-1, 1)$. It is zero-centered but still suffers from vanishing gradients in saturated regions.
- **ReLU**: Outputs $\max(0, x)$. It mitigates vanishing gradients for positive inputs and enables faster training of deep networks, but may suffer from the dying ReLU problem.

10. Consider a binary classification problem with input–label pairs

$$x \in \mathbb{R}^d, \quad y \in \{0, 1\}.$$

A fully connected neural network with an input dimension of d is trained using two hidden layers of sizes $\frac{d}{2}$ and $\frac{d}{4}$, followed by a single output neuron where all hidden layers use sigmoid activation functions. The output layer uses a sigmoid activation, and the loss function is binary cross-entropy.

The network is trained using gradient-based optimization under the following initialization and training settings:

1. All weights and biases are initialized to the same constant value.
2. Xavier initialization is used for all layers.
3. Weights are initialized randomly with small independent noise.

(a) For each initialization setting, determine whether the network is able to learn meaningful representations.

(b) For each case, explain the effect of the initialization on symmetry breaking and gradient flow during training.

Solution:

General principle. For a neural network to learn meaningful representations, neurons within the same layer must receive distinct gradient updates. If neurons start with identical parameters and experience identical gradients, they remain identical throughout training, preventing feature diversity. In addition, stable gradient flow is required to avoid vanishing gradients in deep networks.

1. Same constant initialization

All neurons within a layer are initialized with identical weights and biases. Consequently, they produce identical activations for any input and receive identical gradients during backpropagation. This symmetry is preserved throughout training.

Since all hidden layers use sigmoid activations, gradients are non-zero but identical across neurons. Each layer therefore behaves as a single effective neuron, severely limiting the representational capacity of the network.

Outcome: The network fails to learn meaningful representations and behaves similarly to a shallow model.

Outcome: The network can learn to some extent, but representations remain highly correlated and convergence is slow.

2. Xavier initialization

Xavier initialization assigns independent random weights with variance chosen to preserve activation and gradient variance across layers. This breaks symmetry at initialization and mitigates vanishing gradients in sigmoid networks.

Each neuron receives distinct inputs and gradients, enabling the network to learn diverse features across layers.

Outcome: The network learns meaningful hierarchical representations and trains stably.

3. Random initialization

Random initialization with small independent noise breaks symmetry between neurons. However, without proper variance scaling, activations may saturate the sigmoid function, leading to vanishing gradients.

Learning may occur, but training dynamics are sensitive to the scale of initialization and may be unstable in deeper networks.

Outcome: The network may learn, but convergence is less reliable compared to Xavier initialization.

Conclusion. Constant initialization fails due to symmetry preservation. Properly scaled random initialization such as Xavier initialization provides both symmetry breaking and stable gradient flow, enabling effective learning.

11. Implement a two-layer neural network with 2 input neurons, 10 hidden neurons, and 1 output neuron using ReLU activation in the hidden layer to solve the XOR classification problem. Investigate how the learning rate and bias initialization affect the dying ReLU phenomenon.Weights are initialized as

$$W_{ij} \sim \mathcal{N}(0, 0.01)$$

unless stated otherwise.

Tasks

1. Train the network under the following five configurations:
   - Case 1: Learning rate = 0.1, Bias initialization = -5.0, Epochs = 500
   - Case 2: Learning rate = 0.1, Bias initialization = 0.0, Epochs = 500
   - Case 3: Learning rate = 0.01, Bias initialization = -5.0, Epochs = 500
   - Case 4: Learning rate = 0.01, Bias initialization = 0.0, Epochs = 500
   - Case 5: Learning rate = 0.01, Bias initialization = random, Epochs = 10000
2. For each configuration, plot a grid of figures showing:
   - Row 1: Training loss versus epochs
   - Row 2: Fraction of dead hidden neurons versus epochs
   - Row 3: Classification accuracy versus epochs
3. Answer the following questions:
   (a) Which configurations exhibit dying ReLU behavior? Explain why.
   (b) Why are dead ReLU neurons unable to recover through gradient descent?

12. **Basics of Computational Graphs**

Consider the function

$$z = (x + y)\, y$$

where $x, y \in \mathbb{R}$.

(a) **Graph Representation**
Write the sequence of intermediate computations needed to evaluate z. (These define the computational graph nodes.)

(b) **Forward Evaluation**
Compute z for:

$$x = 2, \quad y = 3$$

(c) **Gradient Computation**

Using the computational graph, compute:

$$\frac{\partial z}{\partial x}, \quad \frac{\partial z}{\partial y}$$

(d) **Conceptual**

In one or two sentences, explain why computational graphs are useful for training neural networks.

Solution:

(a) **Intermediate Computations**

Define:

$$u = x + y$$
$$z = u \cdot y$$

These steps form the computational graph.

(b) **Forward Evaluation**

$$u = 2 + 3 = 5$$
$$z = 5 \times 3 = 15$$

$$\boxed{z = 15}$$

(c) **Gradient Computation**

From $z = uy$:

$$\frac{\partial z}{\partial u} = y \quad , \quad \frac{\partial z}{\partial y} = u$$

From $u = x + y$:

$$\frac{\partial u}{\partial x} = 1 \quad , \quad \frac{\partial u}{\partial y} = 1$$

Chain rule:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial u}\frac{\partial u}{\partial x} = y = 3$$

$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial u}\frac{\partial u}{\partial y} + \left.\frac{\partial z}{\partial y}\right|_{\text{direct}} = y + u = 3 + 5 = 8$$

$$\boxed{\frac{\partial z}{\partial x} = 3, \quad \frac{\partial z}{\partial y} = 8}$$

(d) **Conceptual**

Computational graphs break complex computations into simple steps and enable automatic gradient computation using backpropagation. This makes training neural networks efficient.